

Отчет по лабораторной работе №2

**Неполнодоступная двухсервисная модель Эрланга с
одинаковыми интенсивностями обслуживания и
зарезервированной емкостью**

Иванов Сергей Владимирович, НПИбд-01-23

Содержание

Задание 1	3
1. Алгоритм расчета вероятностных характеристик модели	3
Задание 2	5
Задание 3	8
Задание 4	10
Вывод	12

Задание 1

1. Алгоритм расчета вероятностных характеристик модели

Шаг 1: Ввод исходных данных

C — общая емкость системы;

g — полnodоступная часть емкости;

$C - g$ — часть емкости, зарезервированная для запросов первого типа;

λ_1 — интенсивность поступления запросов первого типа;

λ_2 — интенсивность поступления запросов второго типа;

μ — интенсивность обслуживания.

Шаг 2: Расчет интенсивностей предложенной нагрузки

$$\rho_1 = \frac{\lambda_1}{\mu}, \quad \rho_2 = \frac{\lambda_2}{\mu}.$$

Шаг 3: Расчет вероятности нулевого состояния

Вероятность p_0 определяется из условия нормировки стационарного распределения:

$$p_0 = \left[\sum_{n=0}^g \frac{(\rho_1 + \rho_2)^n}{n!} + (\rho_1 + \rho_2)^g \sum_{n=g+1}^C \frac{\rho_1^{n-g}}{n!} \right]^{-1}.$$

Шаг 4: Расчет стационарных вероятностей состояний

Для состояний от 0 до g запросы обоих типов могут поступать в систему:

$$p_n = p_0 \frac{(\rho_1 + \rho_2)^n}{n!}, \quad n = 0, 1, \dots, g.$$

Для состояний от $g + 1$ до C запросы второго типа блокируются, поэтому учитывается только нагрузка первого типа:

$$p_n = p_0 \frac{(\rho_1 + \rho_2)^g \rho_1^{n-g}}{n!}, \quad n = g + 1, \dots, C.$$

Шаг 5: Расчет вероятностей блокировки

Запрос первого типа блокируется только при полном заполнении системы:

$$E_1 = p_C.$$

Запрос второго типа блокируется при занятости g и более каналов:

$$E_2 = p_g + p_{g+1} + \dots + p_C.$$

Шаг 6: Расчет среднего числа запросов в системе

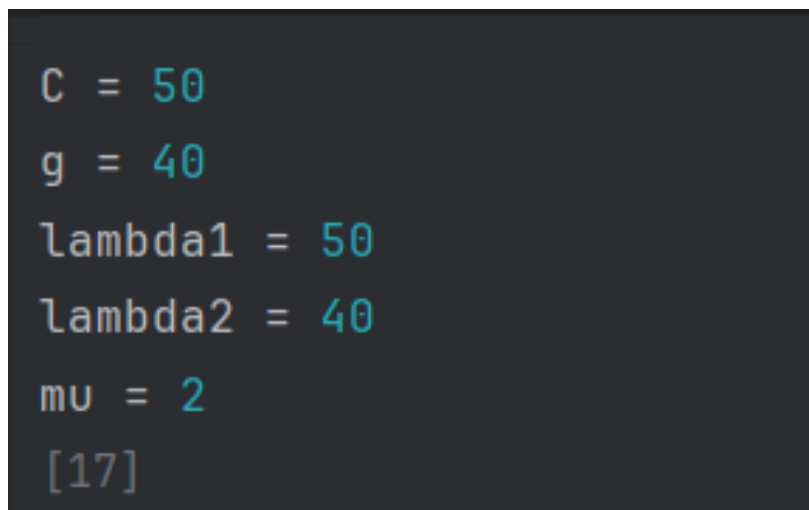
$$N = 0 \cdot p_0 + 1 \cdot p_1 + 2 \cdot p_2 + \dots + C \cdot p_C.$$

Задание 2

Программа реализует алгоритм расчета вероятностных характеристик неполнодоступной двухсервисной модели Эрланга с зарезервированной емкостью. Для заданных параметров рассчитываются интенсивности предложенной нагрузки, вероятность нулевого состояния, стационарное распределение вероятностей, вероятности блокировки запросов первого и второго типов и среднее число обслуживаемых запросов.

В программе используются следующие исходные значения:

$$C = 10, \quad g = 7, \quad \lambda_1 = 4, \quad \lambda_2 = 3, \quad \mu = 1.$$



```
C = 50
g = 40
lambda1 = 50
lambda2 = 40
mu = 2
[17]
```

Рисунок 1: Исходные значения C , g , λ_1 , λ_2 и μ

Основные вычисления выполняются функцией `calculate_model`. Вначале

рассчитываются ρ_1 и ρ_2 , затем непосредственно по формулам определяется p_0 и все вероятности p_n . После этого рассчитываются E_1 , E_2 и N .

```
def calculate_model(C, g, lambda1, lambda2, mu):
    rho1 = lambda1 / mu # интенсивность нагрузки
    rho2 = lambda2 / mu

    p0 = 1 / (
        sum((rho1 + rho2) ** n / math.factorial(n) for n in range(g + 1)) +
        (rho1 + rho2) ** g *
        sum(rho1 ** (n - g) / math.factorial(n) for n in range(g + 1, C + 1))
    )

    p = []

    for n in range(C + 1):
        if n <= g:
            pn = p0 * (rho1 + rho2) ** n / math.factorial(n)
        else:
            pn = p0 * (rho1 + rho2) ** g * rho1 ** (n - g) / math.factorial(n)

        p.append(pn)

    p = np.array(p)

    E1 = p[C] # вероятность бл. заявки
    E2 = sum(p[g:C + 1])
    N = sum(n * p[n] for n in range(C + 1)) # ср. кол-во заявок которые одновременно находятся в системе

    return p, E1, E2, N
```

Рисунок 2: Функция calculate_model

После вызова функции программа выводит распределение вероятностей состояний системы и основные вероятностные характеристики.

```
p, E1, E2, N = calculate_model(C, g, lambda1, lambda2, mu)

for n in range(C + 1):
    print(f"p[{n}] = {p[n]:.6f}")

print()
print(f"E1 = {E1:.6f}")
print(f"E2 = {E2:.6f}")
print(f"N = {N:.6f}")
print(f"Сумма p = {sum(p)}")

[19]
p[43] = 0.030753
p[44] = 0.017473
p[45] = 0.009707
p[46] = 0.005276
p[47] = 0.002806
p[48] = 0.001462
p[49] = 0.000746
p[50] = 0.000373

E1 = 0.000373
E2 = 0.356091
N = 37.868857
Сумма p = 0.9999999999999999
```

Рисунок 3: Распределение вероятностей и значения E1, E2 и N

Для заданных исходных данных получены следующие результаты:

$$E_1 = 0.018406,$$

$$E_2 = 0.375029,$$

$$N = 5.801286.$$

1. Распределение вероятностей $p(n)$. Вероятности состояний возрастают до максимального значения при $n = 6$ и $n = 7$, где $p_n \approx 0.2071$, после чего начинают уменьшаться. После заполнения полнодоступной части $g = 7$ новые запросы второго типа больше не принимаются, поэтому для состояний выше g распределение определяется только поступлением запросов первого типа. Сумма всех вероятностей равна 1, что подтверждает правильность нормировки.

2. Вероятности блокировки. Значение $E_1 = 0.0184$, то есть вероятность блокировки запроса первого типа составляет примерно 1.84%. Первый тип блокируется только при полном заполнении всех $C = 10$ каналов. Значение $E_2 = 0.3750$, то есть вероятность блокировки запроса второго типа составляет примерно 37.50%. Второй тип блокируется уже при занятости $g = 7$ и более каналов, так как не имеет доступа к зарезервированной части емкости.

3. Среднее число запросов. Получено $N = 5.8013$. Это означает, что при заданных параметрах в системе в среднем одновременно обслуживается около 5.8 запроса. При общей емкости $C = 10$ средняя занятость системы составляет около 58%.

Задание 3

Для исследования зависимости вероятностей блокировки от интенсивности предложенной нагрузки первого типа значение ρ_1 изменяется от 0 до 15 с шагом 0.1. Параметры C , g , λ_2 и μ остаются постоянными.

Для каждого значения ρ_1 заново рассчитываются вероятности блокировки E_1 и E_2 , после чего полученные значения сохраняются в списках E1_values и E2_values.

```
rho1_values = np.arange(0, 25, 0.1)
E1_values = []
E2_values = []

for rho1 in rho1_values:
    p, e1, e2, n = calculate_model(C, g, rho1 * mu, lambda2, mu)
    E1_values.append(e1)
    E2_values.append(e2)

plt.figure(figsize=(8, 5))
plt.plot(rho1_values, E1_values, label="E1")
plt.plot(rho1_values, E2_values, label="E2")
plt.xlabel("Нагрузка первого типа  $\rho_1$ ")
plt.ylabel("Вероятность блокировки")
plt.title("Зависимость вероятности блокировки от  $\rho_1$ ")
plt.grid()
plt.legend()
plt.show()
```

Рисунок 1: Расчет E1 и E2 при изменении нагрузки первого типа

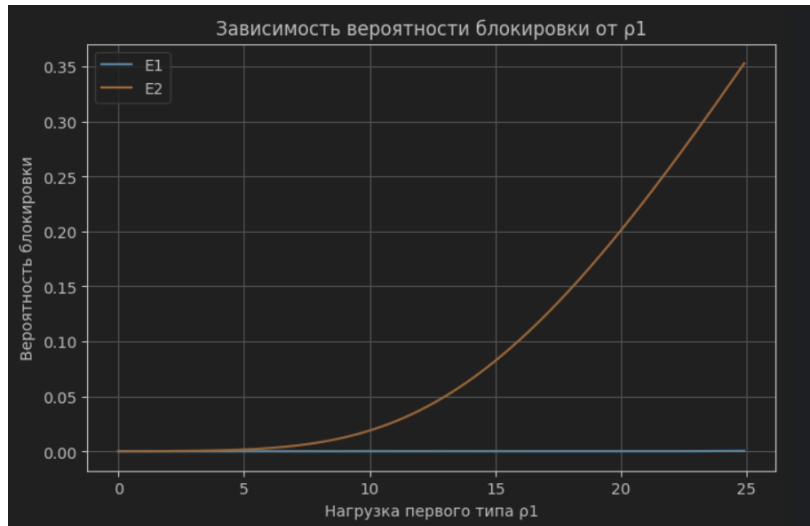


Рисунок 2: График зависимостей E_1 и E_2 от нагрузки первого типа

С увеличением нагрузки первого типа система становится более загруженной, поэтому вероятность блокировки обоих типов увеличивается. Вероятность E_2 на всем диапазоне выше E_1 , поскольку запросы второго типа перестают приниматься уже после заполнения полнодоступной части g , тогда как запросы первого типа могут использовать всю емкость C . При больших значениях ρ_1 система все чаще находится в состояниях с большим числом занятых каналов, поэтому обе вероятности блокировки возрастают.

Задание 4

Для исследования зависимости среднего числа обслуживаемых запросов от интенсивности поступления запросов первого типа значение λ_1 изменяется от 0 до 15 с шагом 0.1. Остальные параметры системы остаются постоянными.

Для каждого значения λ_1 рассчитывается среднее число запросов N , после чего полученные значения сохраняются в списке `N_values`.

```
lambda1_values = np.arange(0, 50.1, 0.1)
N_values = []

for value in lambda1_values:
    _, _, _, n_mean = calculate_model(C, g, value, lambda2, mu)
    N_values.append(n_mean)

plt.figure(figsize=(8, 5))
plt.plot(lambda1_values, N_values)
plt.xlabel("Интенсивность  $\lambda_1$ ")
plt.ylabel("Среднее число запросов N")
plt.title("Зависимость N от  $\lambda_1$ ")
plt.grid()
plt.show()
```

Рисунок 1: Расчет среднего числа запросов N

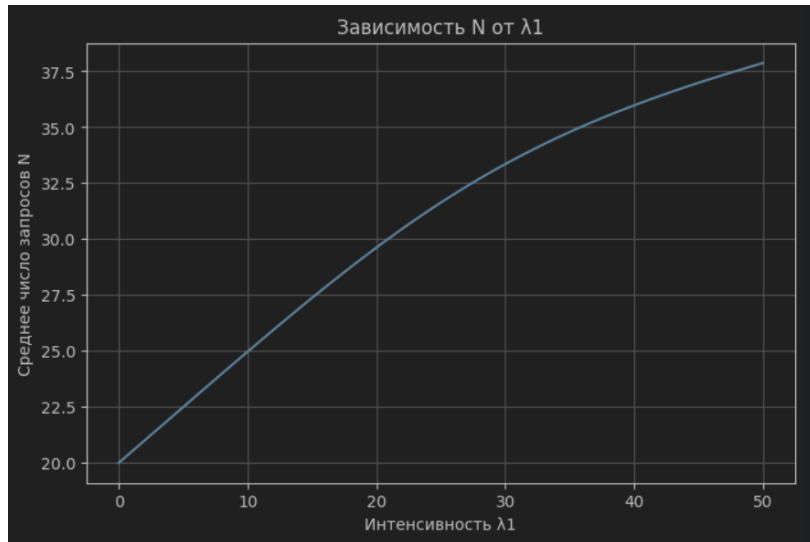


Рисунок 2: График зависимости среднего числа запросов от λ_1

Среднее число обслуживаемых запросов увеличивается с ростом интенсивности поступления запросов первого типа. При малой интенсивности система большую часть времени имеет свободную емкость, поэтому рост λ_1 приводит к заметному увеличению N . При больших значениях λ_1 рост среднего числа запросов замедляется, поскольку число одновременно обслуживаемых запросов не может превышать общую емкость системы $C = 10$.

Вывод

В ходе лабораторной работы была исследована неполнодоступная двухсервисная модель Эрланга с одинаковыми интенсивностями обслуживания и резервированной емкостью.

Для заданных параметров были рассчитаны стационарное распределение вероятностей, вероятности блокировки запросов первого и второго типов и среднее число обслуживаемых запросов. Результаты показывают, что резервирование части емкости для запросов первого типа существенно уменьшает вероятность их блокировки, но одновременно увеличивает вероятность блокировки запросов второго типа.

При увеличении нагрузки первого типа вероятности блокировки E_1 и E_2 возрастают. Среднее число обслуживаемых запросов также увеличивается с ростом интенсивности поступления и при большой нагрузке приближается к максимальной емкости системы C .